

Complete List of SQL Queries with Detailed Explanations

Below is an exhaustive list of all SQL queries used in the SMS, accompanied by detailed explanations of their purposes, structures, and verification steps.

1. Create Database and Tables

```
USE student_db;

CREATE TABLE IF NOT EXISTS users ( id INT
    AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50)
    UNIQUE, password VARCHAR(50), role ENUM('Admin',
    'Teacher', 'Student', 'Parent'), user_id INT, section
    VARCHAR(10)
);

CREATE TABLE IF NOT EXISTS students
    ( id INT PRIMARY KEY, name
    VARCHAR(100), gender
    VARCHAR(10), age INT, section
    VARCHAR(10), science FLOAT,
    english FLOAT, history FLOAT,
    maths FLOAT
);

CREATE TABLE IF NOT EXISTS Teachers ( teacher_id
    INT AUTO_INCREMENT PRIMARY
    KEY, name VARCHAR(100), subject
    VARCHAR(50), section VARCHAR(10)
);

CREATE TABLE IF NOT EXISTS grades (
    student_id INT PRIMARY KEY,
    science_grade CHAR(1),
    english_grade CHAR(1),
    history_grade CHAR(1), maths_grade
    CHAR(1),
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

```

CREATE TABLE IF NOT EXISTS Messages ( id
    INT AUTO_INCREMENT PRIMARY KEY,
    sender_id INT, sender_role
    ENUM('Teacher', 'Student'), receiver_id
    INT,
    receiver_role ENUM('Teacher', 'Student'),
    message TEXT, sent_date DATETIME DEFAULT
    CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS Alerts ( id INT
    AUTO_INCREMENT PRIMARY KEY,
    student_id INT, message TEXT, sent_date DATETIME
    DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES students(id)
);

CREATE TABLE IF NOT EXISTS comments
( id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT, comment TEXT, date
    DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES students(id) );

```

Explanation: This comprehensive query set initializes the student_db database and establishes all required tables. The users table leverages AUTO_INCREMENT for unique id generation and employs an ENUM type to restrict role values, ensuring data consistency and security. The students table stores detailed student records, while the grades table links to it via a foreign key (student_id) to maintain grade data integrity. The Teachers table captures educator details, and the Messages, Alerts, and comments tables handle communication, notifications, and feedback, respectively, with automatic timestamping via DEFAULT CURRENT_TIMESTAMP. The IF NOT EXISTS clause prevents errors if tables are recreated, and foreign keys enforce relational integrity across tables.

Verification: We executed SHOW TABLES; to confirm the creation of all tables and used DESCRIBE <table>; for each table to verify column names, data types, constraints (e.g., primary keys, foreign keys), and default values, ensuring the database structure aligned with the design specifications.

2. Insert Default Admin User

```

INSERT INTO users (username, password, role, user_id) VALUES
('admin', 'admin123', 'Admin', 1);

```

Explanation: This query inserts a default admin user with the username admin, password admin123, role Admin, and user_id of 1, providing an initial entry point for system administration. The user_id serves as a reference for future administrative actions.

Verification: We ran `SELECT * FROM users WHERE username = 'admin';` to verify the record's presence, checking the username, password, role, and user_id fields to ensure accurate insertion.

3. Attempt to Remove Unique Constraint on Teachers.name

```
ALTER TABLE Teachers DROP INDEX IF EXISTS name;
```

Explanation: This query attempts to remove any unique constraint or index on the name column in the Teachers table, addressing a requirement to allow duplicate teacher names (e.g., multiple teachers with the same name in different sections). The IF EXISTS clause was intended to prevent errors if no such index exists.

Verification: The query failed with Error Code: 1064 due to a syntax error (IF EXISTS is not supported in some MySQL versions for DROP INDEX), prompting us to adopt a more robust method to handle the constraint removal.

4. Check and Remove Index on Teachers.name

```
SHOW INDEX FROM Teachers;

SET @index_exists = (SELECT COUNT(*) FROM
                     INFORMATION_SCHEMA.STATISTICS
                     WHERE TABLE_SCHEMA = 'student_db'
                     AND TABLE_NAME = 'Teachers'
                     AND COLUMN_NAME = 'name'
                     AND INDEX_NAME != 'PRIMARY');
SET @drop_sql = IF(@index_exists > 0, 'ALTER TABLE Teachers
DROP INDEX name', 'SELECT "No index to drop"');
PREPARE stmt FROM @drop_sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

Explanation: This multi-step query provides a safe mechanism to check for and remove an index on Teachers.name. The `SHOW INDEX FROM Teachers;` command lists existing indexes, while the `INFORMATION_SCHEMA.STATISTICS` query determines if an index on name exists (excluding the primary key). If an index is found, a dynamic `ALTER TABLE` statement drops it; otherwise, a harmless `SELECT` is executed. This approach avoids errors and ensures flexibility.

Verification: We ran `SHOW INDEX FROM Teachers;` before and after the query to confirm the absence of a name index post-execution, validating that duplicate names are now permissible.

5. Final Attempt to Drop Index

```
ALTER TABLE Teachers DROP INDEX name;
```

Explanation: As a final precautionary step, this query manually attempts to drop the name index, ensuring no residual constraints prevent duplicate entries in the Teachers table.

Verification: The query failed with Error Code: 1091 ("Can't DROP 'name'; check that column/key exists"), confirming that no index remained after the previous step, thus allowing duplicate teacher names as intended.

6. Verify Teachers Table

```
SHOW CREATE TABLE Teachers;
```

Explanation: This query retrieves the full CREATE TABLE statement for the Teachers table, allowing us to inspect its structure and confirm the absence of any unique constraints or indexes on the name column.

Verification: The output displayed the table definition, and we confirmed the absence of a UNIQUE KEY or index on name, ensuring the table structure supports duplicate teacher names.

7. Retrieve Student Data

```
SELECT s.*, g.science_grade, g.english_grade,  
       g.history_grade, g.maths_grade  
FROM students s  
LEFT JOIN grades g ON s.id = g.student_id WHERE section = %s;
```

Explanation: This query retrieves comprehensive student data, including all fields from the students table (s.*) and corresponding grades from the grades table, for a specific section. The

LEFT JOIN ensures that students without grades are included (with NULL values), and the WHERE clause with %s enables dynamic section filtering via parameterized queries, enhancing security against SQL injection.

Verification: We tested the query within the application by passing a section value (e.g., "A") and compared the results against the students and grades tables, verifying that all data, including grades, was accurately retrieved and matched.

8. Average Scores

```
SELECT AVG(science), AVG(english), AVG(history), AVG(maths)  
FROM students WHERE id = %s;
```

Explanation: This query calculates the average scores for each subject (science, english, history, maths) for a specific student identified by their id. The result is used in the Student portal to provide a baseline for predictive analytics, helping students and educators identify performance trends.

Verification: We executed the query with a sample student ID, manually computed the averages from the raw data, and confirmed the query's results were consistent, ensuring accurate aggregation.

9. Section-wise Report

```
SELECT section, AVG(science), AVG(english), AVG(history),  
AVG(maths) FROM students GROUP BY section;
```

Explanation: This query generates a detailed report of average subject scores across all sections, grouping the data by section and computing the mean for each subject. It is a critical tool in the Admin portal for assessing overall academic performance and identifying sectionspecific trends.

Verification: We ran the query and cross-checked the grouped averages against the raw data in the students table, ensuring the calculations were correct and reflective of the actual student scores.

10. Gender Distribution

```
SELECT gender, COUNT(*) FROM students WHERE section = %s GROUP  
BY gender;
```

Explanation: This query counts the number of students by gender within a specific section, grouping the results by gender and using COUNT(*) to tally the totals. It supports the Teacher portal by providing demographic insights, aiding in classroom planning and analysis.

Verification: We tested the query with a sample section, manually counted the students by gender, and confirmed the query's output matched the actual data, ensuring accuracy in demographic reporting.